

BI-ML2 Strojové učení 2

Ultimátní skripta od základů

Zpracováno z přednášek v present/

2026-06-01

Jak se z těchto skript učít

Tahle skripta jsou napsaná tak, abys je mohl/a číst i kdybys o strojovém učení nevěděl/a vůbec nic. Předmět BI-ML2 navazuje na BI-ML1, ale tady ti všechno potřebné z ML1 vždycky nejdřív rychle připomenou, takže nemusíš nic dohledávat. Postupujeme od nejjednodušších věcí ke složitějším a u každého tématu jde výklad v tomto pořadí: (1) *Proč to vůbec řešíme* (motivace), (2) *Intuice* (co to dělá lidsky řečeno), (3) *Přesná matematika* (definice, vzorce, postupy), (4) *Ke zkoušce* (co se po tobě reálně chce).

Čti aktivně. U každého vzorce si říkej: *co je vstup, co počítám, co je výsledek a k čemu mi to je*. Když narazíš na matici nebo sumu, nelekej se: vždycky to pod tím rozepráš slovy.

* u nadpisu znamená „tohle umět do hloubky“ — přesnou definici, postup, vlastnosti a umět spočítat malý příklad ručně. Ústní zkouška je dvě otázky z předem zveřejněného seznamu, každá za 25 bodů; cílem těchto skript je, abys u tabule dokázal/a téma odvodit, ne jen odříkat.

Značení (nauč se ho hned)

- $\mathbf{x} = (x_1, \dots, x_p)^T$ je **vektor příznaků** (angl. features) jednoho datového bodu; p je počet příznaků (dimenze). Příznak = jeden naměřený údaj (např. výška, počet výskytů slova).
- Y je **vysvětlovaná (cílová) proměnná** — to, co chceme předpovídat. $\hat{Y}$ je naše **predikce**.
- Trénovací množina = N dvojic $(Y_1, \mathbf{x}_1), \dots, (Y_N, \mathbf{x}_N)$, tj. N příkladů, u kterých známe i správnou odpověď.
- $\mathbf{X} \in \mathbb{R}^{N \times p}$ je **datová matice**: jeden řádek = jeden bod, jeden sloupec = jeden příznak.
- $\mathbf{w}$ jsou **váhy (parametry)** modelu, $\hat{\mathbf{w}}$ jejich odhad. w_0 je **intercept / bias** (posun).
- $\mathbf{w}^T \mathbf{x} = \sum_i w_i x_i$ je **skalární součin** (sečtu po složkách součiny). $\|\mathbf{x}\| = \sqrt{\mathbf{x}^T \mathbf{x}}$ je **délka (norma)** vektoru.
- $\mathbb{E}$ je **střední hodnota** (průměrná hodnota náhodné veličiny), $\mathbb{P}$ je **pravděpodobnost**.
- $\arg \max_y f(y)$ = ta hodnota y , pro kterou je $f(y)$ největší (ne maximum samo, ale *kde* ho nabývá).
- A^T je **transpozice** (překlopení matice/vektoru), A^{-1} **inverze**, I **jednotková matice**.
- $\propto$ znamená „úměrné“, tj. rovnost až na násobek konstantou, na které zrovna nezáleží.

1 Co musíš umět z ML1 (rychlý odpich)

Než se pustíme do nových věcí, shrňme si jazyk, kterým celý kurz mluví. Bez tohohle by zbytek nedával smysl.

1.1 Co je supervizované učení *

Supervizované (učení s učitelem) znamená, že máme trénovací data, kde u každého bodu $\mathbf{x}_i$ známe i správnou odpověď Y_i . Model se z těchto dvojic „naučí“ funkci f , která pro nový, dosud neviděný bod $\mathbf{x}$ vrátí predikci $\hat{Y} = f(\mathbf{x})$. Dva základní typy úloh:

- Regrese**: Y je spojitě číslo (cena bytu, teplota). Predikujeme číslo.
- Klasifikace**: Y je z konečné množiny tříd (spam/ham, číslice 0–9). Predikujeme třídu.

Proti tomu stojí **nesupervizované učení** (bez správných odpovědí, např. shlukování, redukce dimenze) a **posilované učení** (agent se učí ze sebou získané odměny — viz poslední kapitola).

1.2 Přeučení, trénovací/validační/testovací množina *

Přeučení (overfitting) je situace, kdy se model naučí trénovací data „nazpaměť“ včetně náhodného šumu, ale na nových datech selhává.

Opak je **podučení (underfitting)** — model je moc jednoduchý a nezachytí ani skutečnou strukturu.

- Trénovací množina**: na ní hledáme parametry $\mathbf{w}$.
- Validační množina**: na ní ladíme *hyperparametry* (např. sílu regularizace) a rozhodujeme, kdy přestat trénovat.
- Testovací množina**: sáhneme na ni až úplně na konci, abychom čestně změřili výkon.

Hyperparametr = nastavení, které neučíme z dat gradientem, ale volíme my (např. λ , počet sousedů k , velikost sítě).

1.3 Co je gradient *

Gradient $\nabla_{\mathbf{w}} J(\mathbf{w})$ funkce J je vektor parciálních derivací — ukazuje směr, kterým J *nejrychleji roste*. Proto když chceme J *minimalizovat* (a J je typicky chyba modelu), jdeme proti gradientu. To je celá myšlenka **gradientního sestupu**: $\mathbf{w} \leftarrow \mathbf{w} - \epsilon \nabla_{\mathbf{w}} J(\mathbf{w})$, kde ϵ je velikost kroku. K tomuhle se mnohokrát vrátíme.

1.4 Prokletí dimenzionality *

Intuice: V mnoha rozměrech jsou všechny body „daleko od sebe“ a data jsou neuvěřitelně řídká. Konkrétně: vezmi jednotkovou krychli v d rozměrech. V podkrychli o hraně $(1 - \epsilon)$ je jen $(1 - \epsilon)^d \cdot 100\%$ bodů. Pro velké d jde toto číslo k nule — skoro všechny body jsou v tenké slupce u povrchu. Např. pro $d = 100$ je v okrajové slupce tloušťky 0,01 celých 86,7% bodů.

Důsledek pro ML: s rostoucí dimenzí p potřebujeme *exponenciálně* víc dat, aby body nebyly osamělé. Pro 100 příznaků bys potřeboval 10^{100} bodů, aby byly cca 0,1 od sebe (pro srovnání, atomů ve vesmíru je „jen“ $\sim 10^{80}$). Predikce v řídkém prostoru jsou nespolehlivé, protože model musí extrapolovat. Tohle je hlavní motivace pro *redukci dimenzionality* (kap. 4).

Ke zkoušce: Umět říct, co je gradient a kterým směrem se jde při minimalizaci; vysvětlit overfitting a roli validační množiny; popsat prokletí dimenzionality i s tím příkladem krychle.

2 Regrese, báze funkce a jádrový trik

Tahle kapitola je odrazový můstek. Připomeneme lineární regresi (úplný základ), ukážeme trik s bázeovými funkcemi (jak z lineárního modelu udělat nelineární), a pak odvodíme *jádrový trik*, který je srdcem metody podpůrných vektorů z další kapitoly.

2.1 Lineární regrese *

Model: hodnotu Y v bodě s příznaky $x_1, \dots, x_p$ modelujeme jako vážený součet plus šum:

$$Y = w_0 + w_1 x_1 + \dots + w_p x_p + \varepsilon,$$

kde $w_0, \dots, w_p$ jsou neznámé váhy a ε je náhodná chyba se střední hodnotou $\mathbb{E}\varepsilon = 0$ (v průměru se neplete na žádnou stranu).

Trik s konstantním příznakem: zavedeme $x_0 = 1$ a píšeme $\mathbf{x} = (x_0, \dots, x_p)^T$, $\mathbf{w} = (w_0, \dots, w_p)^T$. Pak je celý model krásně krátce

$$Y = \mathbf{w}^T \mathbf{x} + \varepsilon, \quad \hat{Y} = \hat{\mathbf{w}}^T \mathbf{x}.$$

Intuice: Intercept w_0 je „kam to posadíme“, váhy w_i jsou „o kolik se predikce změní, když x_i vzroste o 1“.

Jak najít váhy — metoda nejmenších čtverců (OLS): chceme, aby predikce seděly co nejlíp. Měříme to **reziduálním součtem čtverců** (součet kvadrátů chyb na trénovacích datech):

$$\text{RSS}(\mathbf{w}) = \sum_{i=1}^N (Y_i - \mathbf{x}_i^T \mathbf{w})^2 = \|\mathbf{Y} - \mathbf{X}\mathbf{w}\|^2.$$

Hledáme $\mathbf{w}$, kde je RSS nejmenší. Minimum poznáme tak, že gradient je nulový ($\nabla \text{RSS} = 0$), což dá tzv. **normální rovnici** $\mathbf{X}^T \mathbf{Y} - \mathbf{X}^T \mathbf{X} \mathbf{w} = 0$. Pokud je $\mathbf{X}^T \mathbf{X}$ invertovatelná (regulární), má jediné řešení:

$$\hat{\mathbf{w}}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}.$$

Když $\mathbf{X}^T \mathbf{X}$ regulární není (např. příznaků je víc než bodů), použijeme pseudoinverzi nebo radši regularizaci — viz dále.

2.2 Hřebenová (ridge) regrese *

Motivace: velké váhy = model je „nervózní“ a snadno se přeučí. Přidáme proto k chybě *penalizaci za velikost vah*:

$$\text{RSS}_\lambda(\mathbf{w}) = \|\mathbf{Y} - \mathbf{X}\mathbf{w}\|^2 + \lambda \sum_{i=1}^p w_i^2.$$

Hyperparametr $\lambda \geq 0$ řídí sílu trestu: $\lambda = 0$ je obyčejné OLS, velké λ tlačí váhy k nule. (Intercept w_0 se obvykle netrestá — proto v maticovém zápisu používáme I' , což je jednotková matice s nulou v levém horním rohu.) Řešení existuje a je jednoznačné pro každé $\lambda > 0$:

$$\hat{\mathbf{w}}_\lambda = (\mathbf{X}^T \mathbf{X} + \lambda I')^{-1} \mathbf{X}^T \mathbf{Y}.$$

Intuice: Přidáním λ na diagonálu „zpevníme“ matici, takže jde vždycky invertovat — proto „hřebenová“ (ridge).

2.3 Lineární model báзовých funkcí *

Problém: lineární regrese umí jen rovné vztahy. Co když je vztah zakřivený?

Řešení: nahradíme původní příznaky jejich *transformacemi*. Zvolíme M **báзовých funkcí** $\varphi_1, \dots, \varphi_M$ (každá bere bod $\mathbf{x}$ a vrací číslo) a model postavíme jako lineární v *těchto nových příznacích*:

$$Y = w_1 \varphi_1(\mathbf{x}) + \dots + w_M \varphi_M(\mathbf{x}) + \varepsilon = \mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) + \varepsilon,$$

kde $\boldsymbol{\varphi}(\mathbf{x}) = (\varphi_1(\mathbf{x}), \dots, \varphi_M(\mathbf{x}))^T$. Klíčové: model je pořád lineární v parametrech $\mathbf{w}$ (takže ho umíme natrénovat stejně snadno), ale jako funkce $\mathbf{x}$ může být libovolně zakřivený.

Typické volby báзовých funkcí:

- $\varphi(\mathbf{x}) = x_i$ — původní příznaky (klasický lineární model).
- $\varphi(\mathbf{x}) = x_i^2, x_k x_\ell$ — mocniny a součiny (polynomiální regrese).
- $\varphi(\mathbf{x}) = \log x_i, \sqrt{x_i}, \sin x_i$ — nelineární transformace.
- $\varphi(\mathbf{x}) = \mathbf{1}_{(a,b)}(x_i)$ — indikátory (1 uvnitř intervalu, jinak 0); umožní modelovat po kouscích.
- $\varphi(\mathbf{x}) = h(\|\mathbf{x} - \mathbf{x}_i\|)$ — **radiální báзовé funkce** centrované v trénovacích bodech.

Trénování je analogické (matici $\mathbf{X}$ nahradíme maticí Φ , kde $\Phi_{ij} = \varphi_j(\mathbf{x}_i)$):

$$\hat{\mathbf{w}}_\lambda = (\Phi^T \Phi + \lambda I)^{-1} \Phi^T \mathbf{Y}, \quad \hat{Y} = \hat{\mathbf{w}}_\lambda^T \boldsymbol{\varphi}(\mathbf{x}).$$

Pozor: Když nevíme nic o systému, bereme typicky *hodně* báзовých funkcí a spoléháme na regularizaci, aby se to nepřeučilo.

2.4 Duální reprezentace *

Tohle je chytrý přepis stejné úlohy, ze kterého vypadne jádrový trik. Místo abychom hledali váhy $\mathbf{w}$ (kterých je M — tolik co báзовých funkcí), budeme je hledat ve tvaru kombinace trénovacích bodů:

$$\mathbf{w} = \Phi^T \boldsymbol{\alpha}, \quad \boldsymbol{\alpha} \in \mathbb{R}^N.$$

Intuice: Tvrdíme: optimální váhy leží v prostoru „napnutém“ trénovacími body. Místo M vah tak hledáme N čísel α (po jednom na trénovací bod). Dosazením do RSS_λ a úpravou dostaneme tzv. duální účelovou funkci. Lze dokázat (Věta v přednášce), že minimum duální i původní úlohy je *stejně*, takže nic neztrácíme.

Gramova matice a jádrová funkce. Definujeme **Gramovu matici** $G = \Phi \Phi^T \in \mathbb{R}^{N,N}$ (je symetrická a pozitivně semidefinitní) a **jádrovou funkci**

$$k(\mathbf{x}, \mathbf{y}) = \boldsymbol{\varphi}(\mathbf{x})^T \boldsymbol{\varphi}(\mathbf{y}).$$

Pak platí $G_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$ — Gramova matice obsahuje hodnoty jádra na všech dvojicích trénovacích bodů. Řešení duální úlohy a predikce vyjdou *jen* přes jádro:

$$\hat{\boldsymbol{\alpha}} = (G + \lambda I)^{-1} \mathbf{Y}, \quad \hat{Y} = \sum_{i=1}^N \hat{\alpha}_i k(\mathbf{x}_i, \mathbf{x}).$$

Intuice: Predikce v novém bodě je vážený součet přes trénovací body, kde váhy říká jádro — „jak moc se nový bod podobá i -tému trénovacímu“. To je hezky interpretovatelné.

2.5 Jádrový trik *

Pozorování: v duální reprezentaci se body objevují *výhradně* přes skalární součiny $\boldsymbol{\varphi}(\mathbf{x})^T \boldsymbol{\varphi}(\mathbf{y})$, které celé umíme nahradit jádrem $k(\mathbf{x}, \mathbf{y})$. To znamená, že **nemusíme báзовé funkce φ vůbec znát ani počítat** — stačí umět spočítat jádro. Tomuto nahrazení se říká **jádrový trik** (kernel trick).

Proč je to skvělé: jádro nám dovolí implicitně pracovat ve velmi vysoké (i nekonečné!) dimenzi, aniž bychom tam cokoliv počítali.

Příklad výpočetní úspory. Regrese nad 1000 obrázky $32 \times 32 = 1024$ pixelů. Kvadratické jádro $k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^2$ odpovídá lineárnímu modelu v prostoru o $525\,825$ báзовých funkcích. Přímou bychom invertovali matici $525\,825 \times 525\,825$. S jádrovým trikem invertujeme jen Gramovu matici 1000×1000 . (Inverze pro odhad $\mathbf{w}$ stojí $O(M^3)$, pro α jen $O(N^3)$.)

Nejdůležitější jádra.

- Lineární:** $k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$ (žádná transformace).
- Polynomiální:** $k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^n$. Pro $p = 2, n = 2$ implicitně vyrobí 6 báзовých funkcí $(1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1 x_2)$.
- Gaussovské (RBF):** $k(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}$. Závisí jen na vzdálenosti bodů; odpovídá nekonečně-dimenzionálnímu prostoru. Nejpoužívanější.

Aby vše fungovalo, musí být jádro **symetrická pozitivně semidefinitní** funkce.

Jádrový model obecně. Hledáme funkci ve tvaru

$f(\mathbf{x}) = \sum_{j=1}^K \alpha_j k(\mu_j, \mathbf{x})$ se středy μ_j . Když jsou středy přímo trénovací body ($f(\mathbf{x}) = \sum_j \alpha_j k(\mathbf{x}_j, \mathbf{x})$), mluvíme o **vektorovém modelu** — přesně to jsme teď odvodili.

Ke zkoušce: Odvodit jádrový trik: proč se v duální úloze body objevují jen ve skalárních součinech a co z toho plyne. Znáť tři jádra (lineární, polynomiální, Gaussovské). Umět říct, proč je duální výpočet $O(N^3)$ místo $O(M^3)$.

3 Lineární klasifikace a metoda podpůrných vektorů (SVM)

Teď přejdeme od regrese ke klasifikaci a pomocí jádrového triku z minulých kapitol dorazíme k jednomu z nejelegantnějších klasifikátorů — **SVM**.

3.1 Diskriminační funkce *

Řešíme **binární klasifikaci**: predikujeme $Y \in \{-1, 1\}$ (dvě třídy, značené $+1$ a -1). Použijeme **lineární diskriminační funkci**

$$f(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) + w_0$$

a predikujeme podle jejího znaménka:

$$\hat{Y}(\mathbf{x}) = \begin{cases} 1 & \text{když } f(\mathbf{x}) \geq 0, \\ -1 & \text{když } f(\mathbf{x}) < 0. \end{cases}$$

Intuice: f je „skóre“. Kladné skóre $\Rightarrow$ třída $+1$, záporné $\Rightarrow$ třída -1 . Hranice mezi třídami je tam, kde $f = 0$.

Geometrie rozhodovací hranice. Množina bodů $\{u : \mathbf{w}^T u + w_0 = 0\}$ je **nadrovina** (v 2D přímka, v 3D rovina, obecně „plochý řez prostorem“). Nazýváme ji **separující nadrovina**. Platí:

- Vektor $\mathbf{w}$ je **normála** (kolmice) k nadrovině — určuje její orientaci.
- Vzdálenost nadroviny od počátku je $d = -w_0 / \|\mathbf{w}\|$.
- Vzdálenost libovolného bodu $\mathbf{x}$ od hranice je $r = f(\mathbf{x}) / \|\mathbf{w}\|$. Je *kladná* na straně, kam míří $\mathbf{w}$, a *záporná* na druhé.

Intuice: Hodnota $f(\mathbf{x})$ až na dělení normou $\|\mathbf{w}\|$ je vzdálenost od hranice se znaménkem. Čím dál od hranice, tím „jistější“ predikce. Odvození vzdálenosti (proč $r = f(\mathbf{x}) / \|\mathbf{w}\|$): každý bod u rozložíme na složku $u_\perp$ kolmou na $\mathbf{w}$ a složku rovnoběžnou s $\mathbf{w}$, kterou napíšeme jako $(d + r) \frac{\mathbf{w}}{\|\mathbf{w}\|}$. Dosadíme do $\mathbf{w}^T u + w_0$; členy z $u_\perp$ a d se vyruší (leží na hranici) a zůstane $\mathbf{w}^T u + w_0 = r \|\mathbf{w}\|$. Tedy $r = (\mathbf{w}^T u + w_0) / \|\mathbf{w}\|$.

3.2 Princip největšího odstupe *

Otázka: jak vybrat $\mathbf{w}$, w_0 ? Předpokládejme zatím, že data jsou **lineárně separabilní** (jdou rozdělit nadrovinou bezchybně). Pak takových nadrovin existuje nekonečně mnoho. Kterou zvolit?

Princip největšího odstupe (largest margin): vyber nadrovinu, která je co nejdál od nejbližších bodů obou tříd. *Intuice:* Široký „pás“ kolem hranice = robustnější, méně náchylné k chybám u nových bodů.

Chceme tedy maximalizovat minimální vzdálenost bodu od hranice, přičemž každý bod má být na správné straně (to znamená $Y_i f(\mathbf{x}_i) > 0$ — znaménko skóre souhlasí se třídou):

$$\max_{\mathbf{w}, w_0} \min_i \frac{Y_i (\mathbf{w}^T \varphi(\mathbf{x}_i) + w_0)}{\|\mathbf{w}\|}.$$

Normalizace škály. Když $\mathbf{w}$, w_0 vynásobíme stejnou konstantou, hranice ani vzdálenosti se nezmění (zlomek se zkrátí). Máme volnost zvolit měřítko — zvolíme ho tak, aby pro nejbližší bod platilo $Y_i (\mathbf{w}^T \varphi(\mathbf{x}_i) + w_0) = 1$. Pak pro všechny body

$$Y_i (\mathbf{w}^T \varphi(\mathbf{x}_i) + w_0) \geq 1,$$

a minimální odstup je přesně $1/\|\mathbf{w}\|$. Maximalizace $1/\|\mathbf{w}\|$ je totéž co minimalizace $\|\mathbf{w}\|^2$. Dostáváme finální úlohu:

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{za podmínky} \quad Y_i (\mathbf{w}^T \varphi(\mathbf{x}_i) + w_0) \geq 1 \quad \forall i.$$

Tohle je **kvadratické programování** (minimalizujeme kvadratickou funkci za lineárních nerovnostních podmínek). Body, kde podmínka platí přesně (s rovností = 1), se nazývají **podpůrné vektory (support vectors)** — jen *ony* určují polohu nadrovin. Odtud název **metoda podpůrných vektorů (SVM)**.

3.3 Lineárně neseparabilní případ a měkký odstup *

V realitě data čistě oddělit nejdou. Povolíme tedy některým bodům být „za čarou“, ale potrestáme to. Zavedeme **uvolněné proměnné (slack)** $\xi_i \geq 0$:

- $\xi_i = 0$: bod je správně a vně pásu (v pořádku), $Y_i f(\mathbf{x}_i) \geq 1$.
- $0 < \xi_i \leq 1$: bod je uvnitř pásu, ale na správné straně.
- $\xi_i > 1$: bod je na špatné straně — chyba klasifikace.

Tvrdou podmínku změkčíme na $Y_i f(\mathbf{x}_i) \geq 1 - \xi_i$ a do účelové funkce přidáme trest za sumu slacků:

$$\min_{\mathbf{w}, w_0, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i, \quad \xi_i \geq 0, \quad Y_i f(\mathbf{x}_i) \geq 1 - \xi_i.$$

Parametr C řídí kompromis (počet tolerovaných chyb). **Pozor:** Možná neintuitivně: *velké C = slabší regularizace* (silně trestáme chyby, model se ohýbá k datům). *Malé C = silná regularizace* (chyby skoro netrestáme, váhy zůstanou malé). Často se bere $C = 1/(\nu N)$, kde $\nu \in (0, 1]$ je akceptovatelný podíl chyb (tzv. ν -SVM).

3.4 Duální formulace a jádrový trik v SVM *

Pomocí Lagrangeových multiplikátorů se úloha přepíše do **duální formulace** — a tady přesně využijeme jádrový trik, protože body se objeví jen ve skalárních součinech. Maximalizujeme

$$\tilde{L}(a) = \sum_{i=1}^N a_i - \frac{1}{2} \sum_{i,j=1}^N a_i a_j Y_i Y_j k(\mathbf{x}_i, \mathbf{x}_j)$$

za podmínek $0 \leq a_i \leq C$ a $\sum_i a_i Y_i = 0$. Řešení dává diskriminační funkci čistě přes jádro:

$$f(\mathbf{x}) = \sum_{j=1}^N a_j Y_j k(\mathbf{x}, \mathbf{x}_j) + w_0, \quad \hat{Y}(\mathbf{x}) = \text{sgn } f(\mathbf{x}).$$

Intuice: Díky jádru SVM klasifikuje i nelineárně oddělitelná data — implicitně přejde do vyšší dimenze, kde už lineární hranice stačí.

Vlastnosti řešení (důležité!).

- Řešení je **řídke**: $a_j = 0$ pro všechny správně klasifikované body *mimo* pás. Predikce tedy závisí jen na podpůrných vektorech.

- Body s $0 < a_j < C$ leží přesně na hranici pásu.
- Body s $a_j = C$ jsou uvnitř pásu nebo špatně klasifikované.
- Čím větší C , tím *méně* podpůrných vektorů (řidší řešení).

Příklad XOR. Lineárně neseparabilní XOR (body „do kříže“) SVM s Gaussovským jádrem vyřeší. C ladí hladkost hranice: malé C (silná regularizace) = hladká hranice s více chybami, velké C = klikatá hranice lpící na datech.

Ke zkoušce: Odvodit úlohu největšího odstupe od geometrie nadrovin až po $\min \frac{1}{2} \|\mathbf{w}\|^2$. Vysvětlit slack proměnné a roli C (a že velké C = slabší regularizace). Vědět, co jsou podpůrné vektory a proč je řešení řídke. Umět napsat diskriminační funkci přes jádro.

4 Redukce dimenzionality

4.1 Motivace a hypotéza variet *

Z prokletí dimenzionality (kap. 1) víme, že vysoká dimenze je problém. Naštěstí: **hypotéza variet (manifold hypothesis)** říká, že reálná mnohorozměrná data ve skutečnosti leží podél *variety* mnohem menší dimenze.

Intuice: Varieta = zakřivená „plocha“ nižší dimenze vnořená do velkého prostoru (např. 2D papír stočený v 3D). Příklad: MNIST jsou obrázky $28 \times 28 = 784$ pixelů, tedy body v 784-rozměrném prostoru. Ale když pixely vyplníme náhodně, skoro nikdy nedostaneme číslici — smysluplné obrázky tvoří malinkou oblast. Další příklad: **švýcarská rolka (swiss roll)** — body v 3D leží na 2D stočené ploše.

Dvě rodiny metod:

- Lineární projekce** (ortogonální projekce na podprostor) — např. PCA. Zachytí jen lineární strukturu.
- Manifold learning** (nelineární) — např. LLE, t-SNE, UMAP. Zachytí i zakřivení.

4.2 Ortogonální projekce *

Cíl: každý bod $\mathbf{x} \in \mathbb{R}^p$ promítnout na q -rozměrný podprostor V ($q < p$), tj. „zploštit“ do menšího prostoru.

Ortonormální báze. Vezmeme bázi $b_1, \dots, b_p$ prostoru $\mathbb{R}^p$, která je *ortonormální* (vektory na sebe kolmé a délky 1), a to tak, že prvních q vektorů naplní podprostor V . Pak každý bod rozložíme jednoznačně na

$$\mathbf{x} = \underbrace{v_{\mathbf{x}}}_{\text{leží ve } V} + \underbrace{u_{\mathbf{x}}}_{\text{kolmé na } V}, \quad \tau_i = \mathbf{x}^T b_i.$$

Souřadnice projekce $v_{\mathbf{x}}$ ve V jsou $t_{\mathbf{x}} = (\tau_1, \dots, \tau_q)^T$. Maticově (sloupce $\mathbf{V} \in \mathbb{R}^{p,q}$ jsou $b_1, \dots, b_q$):

$$t_{\mathbf{x}} = \mathbf{V}^T \mathbf{x}, \quad v_{\mathbf{x}} = \mathbf{V} \mathbf{V}^T \mathbf{x}, \quad \mathbf{V}^T \mathbf{V} = I_q.$$

Pro celý dataset: **transformovaná data** (souřadnice ve V , tj. redukovaná reprezentace) jsou $\mathbf{T}_q = \mathbf{X} \mathbf{V} \in \mathbb{R}^{N,q}$; *zpětně promítnutá* data v původním prostoru jsou $\mathbf{X} \mathbf{V} \mathbf{V}^T$.

Intuice: $\mathbf{V}^T \mathbf{x}$ = „spočti q nových souřadnic“. $\mathbf{V} \mathbf{V}^T \mathbf{x}$ = „a vrať se zpátky do původního prostoru na stín bodu na podprostoru“. Rozdíl $\|\mathbf{x} - v_{\mathbf{x}}\| = \|u_{\mathbf{x}}\|$ je chyba projekce (co jsme zahodili). Z ortogonalit plyne Pythagoras: $\|\mathbf{x}\|^2 = \|v_{\mathbf{x}}\|^2 + \|u_{\mathbf{x}}\|^2$.

4.3 Analýza hlavních komponent (PCA) *

Otázka: který podprostor V zvolit, aby chyba projekce byla nejmenší? Odpověď je PCA.

Krok 1 — středování. Nejdřív od dat odečteme průměr: $\mathbf{x}'_i = \mathbf{x}_i - \bar{\mathbf{x}}$, kde $\bar{\mathbf{x}} = \frac{1}{N} \sum_i \mathbf{x}_i$. *Proč:* minimalizace chyby projekce vede vždy na podprostor procházející těžištěm dat; středování to zařídí a zároveň zaručí nejmenší možnou chybu nezávisle na V .

Krok 2 — varianční (kovarianční) matice. Spočteme

$$\mathbf{S} = \frac{1}{N-1} \mathbf{X}'^T \mathbf{X}', \quad S_{ij} = \widehat{\text{cov}}(X_i, X_j).$$

Tahle symetrická matice $p \times p$ popisuje, jak spolu příznaky kovariují (na diagonále jsou rozptyly).

Krok 3 — vlastní čísla a vektory. $\mathbf{S}$ je symetrická a pozitivně semidefinitní, takže má reálná nezáporná vlastní čísla $\lambda_1 \geq \dots \geq \lambda_p \geq 0$ a ortonormální vlastní vektory $b_1, \dots, b_p$.

Podprostor V vezmeme z prvních q vlastních vektorů (těch s největšími λ). Tomu se říká **analýza hlavních komponent (PCA)**.

Proč to funguje (klíčový výsledek). Součet kvadrátů chyb projekce na V z prvních q vektorů je $(N-1)(\lambda_{q+1} + \dots + \lambda_p)$ — tedy součet *zahozených* vlastních čísel. Žádný jiný q -rozměrný podprostor nedá menší chybu. Ekvivalentně: **minimalizace chyby projekce = maximalizace rozptylu promítnutých dat.**

Hlavní komponenty a jejich statistika. Nové příznaky $T_i = b_i^T X'$ jsou **hlavní komponenty**. Platí:

- Rozptyl i -té komponenty $= \lambda_i$ (vlastní číslo). $\Rightarrow$ vybíráme směry s největším rozptylem.
- Komponenty jsou **nekorelované** (kovariance mezi různými $= 0$).
- Podíl rozptylu „vysvětleného“ prvními q komponentami $= \frac{\lambda_1 + \dots + \lambda_q}{\lambda_1 + \dots + \lambda_p}$.

Intuice: PCA otočí souřadnice tak, aby první osa mířila tam, kde data nejvíc „prší“, druhá kolmo na ni s druhým největším rozptylem atd. Zahodíme osy, kde se skoro nic neděje.

Praktické poznámky.

- Numericky lépe než spektrálním rozkladem S se PCA počítá přes **SVD** (rozklad X' na singulární hodnoty).
- Počet komponent q se volí z „**elbow**“ **grafu** (kumulativní vysvětlený rozptyl vs. q , hledáme „loket“ zlomu).
- Pro $q = p$ nic nezahodíme, jen přejdeme do ortonormální báze (komponenty na sebe kolmé).

4.4 Manifold learning — LLE *

Problém PCA: zachytí jen lineární strukturu. Švýcarskou rolku „rozmáčkne“ a překryje vrstvy. Nelineární metody se místo globálních vlastností soustředí na *lokální*.

Lokálně lineární vnoření (LLE) — myšlenka: každý bod jde dobře popsat jako lineární kombinaci svých sousedů. Tyhle lokální vztahy chceme zachovat i v nízké dimenzi. Dvoufázově:

Fáze 1 — váhy (v původním prostoru). Pro každý bod $\mathbf{x}_i$ najdi k nejbližších sousedů a váhy w_{ij} tak, aby $\mathbf{x}_i$ byl co nejlíp rekonstruován z těch sousedů:

$$W^* = \arg \min_W \sum_{i=1}^N \left\| \mathbf{x}_i - \sum_j w_{ij} \mathbf{x}_j \right\|^2,$$

za podmínek $w_{ij} = 0$ pro ne-sousedy a $\sum_j w_{ij} = 1$. *Intuice:* Váhy kódují „jak bod sedí mezi svými sousedy“ — jeho lokální geometrii.

Fáze 2 — vnoření (v nízké dimenzi). Najdi q -rozměrné body z_i , které ty *stejně* váhy splňují co nejlíp:

$$Z^* = \arg \min_Z \sum_{i=1}^N \left\| z_i - \sum_j w_{ij}^* z_j \right\|^2.$$

Intuice: Hledáme nízkorozměrné rozložení, kde každý bod pořád „sedí mezi sousedy“ stejně jako v originále. Tím rozbálíme rolku do roviny.

Pozor: Když je $k > p$ (více sousedů než dimenzí), jsou sousedi lineárně závislí a je vhodné použít *modifikované LLE*.

Další metody (jen názvy ke zkoušce): MDS, Isomap, t-SNE, UMAP — různé nelineární redukce, hojně na vizualizaci dat do 2D.

Ke zkoušce: Vysvětlit hypotézu variet. Odvodit PCA: středování $\rightarrow$ varianční matice $\rightarrow$ vlastní vektory s největšími λ . Vědět, že rozptyl komponenty = vlastní číslo a komponenty jsou nekorelované. Popsat dvě fáze LLE a čím se liší od PCA.

5 Klasifikace přes pravděpodobnost: naivní Bayes

Ted' úplně jiný přístup ke klasifikaci — pravděpodobnostní. Nepočítáme „skóre“, ale rovnou *pravděpodobnost, že bod patří do třídy*.

5.1 MAP odhad a Bayesova věta *

Příznaky bereme jako náhodný vektor X , třídu jako náhodnou veličinu Y . Kdybychom znali $\mathbb{P}(Y = y | X = \mathbf{x})$ (pravděpodobnost třídy y při pozorovaných příznacích $\mathbf{x}$), predikovali bychom nejpravděpodobnější třídu:

$$\hat{Y} = \arg \max_y \mathbb{P}(Y = y | X = \mathbf{x}).$$

Tomu se říká **MAP odhad** (maximum a posteriori). *Intuice:* „Která třída je při těchtole příznacích nejpravděpodobnější?“

Problém: $\mathbb{P}(Y = y | X = \mathbf{x})$ neumíme snadno odhadnout přímo. Jdeme oklikou přes **Bayesovu větu** (z BI-PST):

$$\mathbb{P}(Y = y | X = \mathbf{x}) = \frac{\mathbb{P}(X = \mathbf{x} | Y = y) \mathbb{P}(Y = y)}{\mathbb{P}(X = \mathbf{x})}.$$

Jmenovatel $\mathbb{P}(X = \mathbf{x})$ je pro všechny třídy stejný, takže při hledání maxima ho zahodíme:

$$\hat{Y} = \arg \max_y \underbrace{\mathbb{P}(X = \mathbf{x} | Y = y)}_{\text{věrohodnost}} \underbrace{\mathbb{P}(Y = y)}_{\text{apriorní}}.$$

$\mathbb{P}(Y = y)$ (jak časté je která třída) odhadneme triviálně z četností. Zbývá $\mathbb{P}(X = \mathbf{x} | Y = y)$ — a to je jádro problému.

5.2 Naivní předpoklad *

Naivní Bayes stojí na předpokladu:

Za podmínky $Y = y$ jsou všechny příznaky nezávislé.

Pak se sdružená pravděpodobnost rozpadne na součin marginálních:

$$\mathbb{P}(X = \mathbf{x} | Y = y) = \prod_{i=1}^p \mathbb{P}(X_i = x_i | Y = y),$$

$$\hat{Y} = \arg \max_y \mathbb{P}(Y = y) \prod_{i=1}^p \mathbb{P}(X_i = x_i | Y = y).$$

Intuice: Místo abychom modelovali, jak příznaky spolupracují (což by chtělo nesmírně dat), modelujeme každý příznak *zvlášť*. „Naivní“ proto, že nezávislost obvykle neplatí.

Proč to funguje i přes špatný předpoklad:

- Separace příznaků = **odolnost vůči prokletí dimenzionality**: každý $\mathbb{P}(X_i = x_i | Y = y)$ odhadneme z mála dat a potřeba dat neroste s počtem příznaků.
- I když je odhad sdružené pravděpodobnosti nepřesný, **MAP odhad** (jen kdo je největší) bývá správný, pokud má skutečná třída nejvyšší skóre. To je častá situace.

Příklad (ruční výpočet). Tři binární příznaky, binární Y . Z trénovací tabulky odhadneme např. $\hat{p}(X_1=0 | Y=1) = \frac{1}{3}$, $\hat{p}(X_2=1 | Y=1) = 1$, $\hat{p}(X_3=1 | Y=1) = \frac{2}{3}$, $\hat{p}(Y=1) = \frac{1}{2}$. Pro bod $\mathbf{x} = (0, 1, 1)$:

$$\frac{1}{3} \cdot 1 \cdot \frac{2}{3} \cdot \frac{1}{2} = \frac{1}{9}, \quad (\text{pro } Y=0) = \frac{1}{27}.$$

$$\frac{1}{9} > \frac{1}{27} \Rightarrow \hat{Y} = 1.$$

5.3 Modely marginálních rozdělání *

Pro každý příznak potřebujeme model $\mathbb{P}(X_i = x_i | Y = y)$. Podle typu příznaku:

(a) Binární příznak — Bernoulliho rozdělení.

$\mathbb{P}(X = 1 | Y = y) = p_y$. MLE odhad (maximální věrohodnost):

$$\hat{p}_y = \frac{N_{1,y}}{N_{1,y} + N_{0,y}} \quad (\text{podíl jedniček ve třídě } y).$$

Pozor: Hrozí **kolaps**: když se hodnota v trénovacích datech třídy y nevyskytne, vyjde odhad 0, celý součin se vynuluje a třídu y pak *nikdy* nepredikujeme. Řešení viz Bayesovský odhad níže.

(b) Kategorický příznak — k hodnot. $\mathbb{P}(X = c_j | Y = y) = p_{j,y}$, MLE $\hat{p}_{j,y} = N_{j,y} / \sum_{\ell} N_{\ell,y}$.

(c) Spojitý příznak — Gaussovo rozdělení. Pro spojitý X je $\mathbb{P}(X = x) = 0$, takže místo pravděpodobnosti použijeme **podmíněnou hustotu** $f_{X|y}(x)$. Typicky normální $N(\mu_y, \sigma_y^2)$ s MLE odhady

$$\hat{\mu}_y = \frac{1}{N_y} \sum_i x_i, \quad \hat{\sigma}_y^2 = \frac{1}{N_y} \sum_i (x_i - \hat{\mu}_y)^2$$

(přes body třídy y). V MAP odhadu pak hustoty a pravděpodobnosti smícháme: diskrétní příznaky přes $\mathbb{P}$, spojitý přes f . (V scikit-learn: `BernoulliNB`, `GaussianNB`.)

5.4 Bayesovský odhad a vyhlazení *

Motivace: chceme se zbavit kolapsu a umět zohlednit „expertní“ apriorní očekávání (např. mince je nejspíš fěrová, i když 3× padl orel).

Bayesovský přístup. Na parametr (např. p u Bernoulliho) se díváme jako na náhodnou veličinu s **apriorním rozdělením** $f_p(p)$ (naše počáteční očekávání). Po pozorování dat ho přepočteme Bayesovou větou na **aposteriorní rozdělení**:

$$f_p(p | x) = \frac{\mathbb{P}(X = x | p) f_p(p)}{\mathbb{P}(X = x)}.$$

Bodový odhad pak vezmeme jako střední hodnotu aposteriorního rozdělení $\hat{p} = \int p f_p(p | x) dp$.

Beta apriorno → **add-one smoothing**. Pro Bernoulliho se bere **Beta** rozdělení (jeho speciální případ je rovnoměrné = „nic nevím“). Výsledný odhad je

$$\hat{p}_y = \frac{N_{1,y} + 1}{N_{1,y} + N_{0,y} + 2}.$$

Tomuhle se říká **add-one smoothing / Laplaceovo pravidlo následnosti**. Na kolaps už netrpí (nikdy nevyjde 0 ani 1). Pro kategorický příznak

$$\text{analogicky } \hat{p}_{j,y} = \frac{N_{j,y} + 1}{\sum_{\ell} N_{\ell,y} + k}.$$

Intuice: Add-one = „předstírej, že jsi každou hodnotu viděl jednou navíc“. Tím se vyhněs jistotám typu „tohle se nikdy nestane“.

Ke zkoušce: Odvodit MAP přes Bayesovu větu a zdůvodnit, proč se zahodí jmenovatel. Vyslovit naivní předpoklad a napsat součinný tvar. Spočítat malý příklad ručně. Vysvětlit kolaps MLE a jak ho add-one smoothing řeší (znát vzorec $\frac{N_{1,y}+1}{N_{0,y}+N_{1,y}+2}$).

6 Generativní vs. diskriminativní modely, LDA

6.1 Dva tábory modelů *

Tohle je důležité pojmové dělení, na které se u zkoušky rádi ptají.

- Generativní přístup:** modelujeme *sdrúženou* pravděpodobnost $\mathbb{P}(X = \mathbf{x}, Y = y) = \mathbb{P}(X = \mathbf{x} | Y = y) \mathbb{P}(Y = y)$. „Generativní“ = máme úplný popis toho, jak data vznikla, a umíme *generovat* nová pozorování. Příklady: naivní Bayes, (Gaussovská) diskriminační analýza.
- Diskriminativní přístup:** modelujeme *přímo* $\mathbb{P}(Y = y | X = \mathbf{x})$, bez modelu pro to, jak vznikly příznaky. Příklady: logistická regrese, neuronové sítě. (V širším smyslu sem patří i přímá predikce Y bez pravděpodobností — třeba rozhodovací stromy.)

Intuice: Generativní = „nauč se popsat každou třídu a pak se ptej, ze které třídy bod nejspíš pochází“. Diskriminativní = „nauč se rovnou kreslit hranici mezi třídami“. Diskriminativní dnes převažují.

6.2 Klasifikace textů (bag-of-words + multinomický model) *

Reálná aplikace naivního Bayese. Dokument reprezentujeme **bag-of-words**: má D příznaků, kde X_j = počet výskytů j -tého slova ze slovníku (pořadí slov ignorujeme). Model:

$$\mathbb{P}(X = \mathbf{x} | Y = y) = \frac{n!}{\prod_j x_j!} \prod_{j=1}^D p_{j,y}^{x_j},$$

kde $p_{j,y}$ = pravděpodobnost, že náhodné slovo dokumentu třídy y je j -té slovo, $n = \sum_j x_j$ = délka dokumentu. To je **multinomické rozdělení**. Odhad (poměr výskytů slova v dané kategorii) a jeho vyhlazená verze:

$$\hat{p}_{j,y} = \frac{N_{j,y}}{N_y}, \quad \hat{p}_{j,y} = \frac{N_{j,y} + 1}{N_y + D} \text{ (add-one)}.$$

(scikit-learn: MultinomialNB.)

6.3 Log-sum-exp trik *

Problém: $\mathbb{P}(X = \mathbf{x} | Y = y)$ je součin mnoha malých čísel → **numerické podtečení** (počítač to zaokrouhlí na nulu). Řešení: počítáme v logaritmech. Pro MAP nám stačí

$$\log \mathbb{P}(Y = y | X = \mathbf{x}) \propto \log \mathbb{P}(X = \mathbf{x} | Y = y) + \log \mathbb{P}(Y = y),$$

(u naivního Bayese je první člen součet logaritmů — bezpečné). Když potřebujeme i jmenovatel (log součtu), použijeme **log-sum-exp trik**: vytkneme největší exponent

$$\log \sum_y e^{-b_y} = -B + \log \sum_y e^{B-b_y}, \quad B = \min_y b_y.$$

Intuice: $\log(e^{-120} + e^{-121}) = \log(e^0 + e^{-1}) - 120$ — místo počítání s mizivými čísly přesuneme měřítko. (SciPy: logsumexp.)

6.4 Gaussovská a kvadratická diskriminační analýza (GDA/QDA) *

Generativní model, kde každou třídu popíšeme **vícerozměrným normálním rozdělením** $X | Y = y \sim N(\mu_y, \Sigma_y)$. Hustota:

$$f_X(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^T \Sigma^{-1}(\mathbf{x} - \mu)\right).$$

μ je vektor středních hodnot (kde je „kopec“), Σ varianční matice (jeho tvar a natočení). MAP: $\hat{Y} = \arg \max_y f_{X|Y}(\mathbf{x}) \mathbb{P}(Y = y)$.

Při porovnávání tříd se v exponentu objeví **kvadratický výraz** $(\mathbf{x} - \mu_y)^T \Sigma_y^{-1}(\mathbf{x} - \mu_y)$ — proto **kvadratická diskriminační analýza (QDA)** a rozhodovací hranice je zakřivená. MLE odhady:

$$\hat{\pi}_y = \frac{N_y}{N}, \quad \hat{\mu}_y = \frac{1}{N_y} \sum_{i: y_i = y} \mathbf{x}_i, \quad \hat{\Sigma}_y = \frac{1}{N_y} \sum_{i: y_i = y} (\mathbf{x}_i - \hat{\mu}_y)(\mathbf{x}_i - \hat{\mu}_y)^T.$$

Pozor: Když je Σ_y *diagonální*, příznaky jsou nezávislé a dostáváme zpět **gaussovský naivní Bayes**.

6.5 Lineární diskriminační analýza (LDA) *

Speciální případ: předpokládáme *stejnou* varianční matici pro všechny třídy, $\Sigma_y = \Sigma$. Pak se kvadratické členy $(\mathbf{x}^T \Sigma^{-1} \mathbf{x})$ napříč třídami vykrátí a v exponentu zůstane **lineární** výraz ve $\mathbf{x}$. Zavedeme-li

$$\gamma_y = -\frac{1}{2} \mu_y^T \Sigma^{-1} \mu_y + \log \pi_y, \quad \beta_y = \Sigma^{-1} \mu_y,$$

pak $\mathbb{P}(Y = y | X = \mathbf{x}) \propto e^{\beta_y^T \mathbf{x} + \gamma_y}$. Rozhodovací hranice mezi dvěma třídami je **nadrovina** (lineární) — odtud „lineární“ diskriminační analýza.

Krásné spojení s logistickou regresí. Pro binární klasifikaci vyjde

$$\mathbb{P}(Y = 1 | X = \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + w_0),$$

kde σ je **logistická sigmoida** (viz dál) a

$$\mathbf{w} = \Sigma^{-1}(\mu_1 - \mu_0), \quad w_0 = -\frac{1}{2} \mathbf{w}^T (\mu_1 + \mu_0) + \log \frac{\pi_1}{\pi_0}.$$

Intuice: LDA vede na stejný předpis jako logistická regrese! Liší se jen tím, jak ho trénujeme — LDA přes generativní MLE (odhad μ, Σ, π), logistická regrese přímo diskriminativně. Pro Σ se v LDA bere společný odhad přes obě třídy.

6.6 Fisherova LDA (FLDA) *

Stejný směr $\mathbf{w}$ jde získat i bez pravděpodobnostních předpokladů — čistě jako **nejlepší směr projekce pro klasifikaci**. **Cíl FLDA:** promítnout data na přímku tak, aby **průměry tříd byly od sebe co nejdál** a zároveň **rozptyl uvnitř tříd byl co nejmenší**. Účelová funkce:

$$J(\mathbf{w}) = \frac{(m_1 - m_0)^2}{s_0^2 + s_1^2} = \frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}},$$

kde $S_B = (\hat{\mu}_1 - \hat{\mu}_0)(\hat{\mu}_1 - \hat{\mu}_0)^T$ je **mezitřídní rozptyl** (jak daleko jsou třídy) a S_W je **vnitrotřídní rozptyl** (jak roztažené jsou uvnitř). Maximum (přes vlastní vektor) dává

$$\mathbf{w} \propto S_W^{-1}(\hat{\mu}_1 - \hat{\mu}_0),$$

což je *totožné* s LDA (neboť $S_W = N\hat{\Sigma}$). *Intuice:* FLDA dává LDA optimalizační smysl; LDA dává FLDA pravděpodobnostní smysl (platí za normality a stejných Σ).

Pro C tříd se FLDA používá jako **redukce dimenze řízená třídami**:

hledáme až $C - 1$ směrů (matici W), které maximalizují $\frac{\det(W^T S_B W)}{\det(W^T S_W W)}$.

Na rozdíl od PCA (nevšímá si tříd) hledá LDA projekci, kde jsou třídy nejlépe oddělené.

Ke zkoušce: Rozlišit generativní vs. diskriminativní (definice + příklady). Vysvětlit, proč QDA dává kvadratickou a LDA lineární hranici (vykrácení kvadratického členu při $\Sigma_y = \Sigma$). Znáť vztah LDA ↔ logistická regrese a FLDA ↔ LDA. Rozdíl PCA vs. LDA jako redukce dimenze.

7 Neuronové sítě I: perceptron

7.1 Inspirace a model neuronu *

Nápad (McCulloch & Pitts, 1943): neuron = jednoduchá výpočetní jednotka, která sečte vstupy, porovná s prahem a vydá výstup. Rosenblatt (1957) z toho udělal trénovatelný **perceptron** (jeden umělý neuron se spojitými vstupy a vahami).

Model perceptronu. Spočteme **vnitřní potenciál** (vážený součet vstupů + bias):

$$\xi = w_0 + \sum_{i=1}^p w_i x_i = \mathbf{w}^T \mathbf{x} + w_0,$$

a aplikujeme **skokovou aktivační funkci**:

$$\hat{Y} = g(\xi), \quad g(\xi) = \begin{cases} 1 & \xi \geq 0, \\ 0 & \xi < 0. \end{cases}$$

Neuron se „aktivuje“ ($g = 1$), když $\sum_i w_i x_i \geq -w_0$; proto se $-w_0$ říká **práh**. Geometricky: váhy určují nadrovinu, která prostor vstupů dělí na dva poloprostory (aktivní / neaktivní). *Intuice:* Perceptron = lineární klasifikátor s tvrdým rozhodnutím 0/1.

7.2 Trénovací algoritmus perceptronu *

Jak najít váhy? Procházíme body trénovací množiny a po každém upravíme váhy podle chyby:

$$\text{error} = Y - \hat{Y}, \quad w_i \leftarrow w_i + \text{error} \cdot x_i, \quad w_0 \leftarrow w_0 + \text{error}.$$

Intuice: Když je predikce správná (error = 0), neměníme nic. Když se pleteme, posuneme nadrovinu směrem ke správné odpovědi. Tomuto se říká **on-line trénování** (update hned po každém bodu). Skončíme, když na žádném bodu nechybujeme.

Věta o konvergenci. Jsou-li data lineárně separabilní (existuje $\mathbf{w}^*$, $\|\mathbf{w}^*\| = 1$, s odstupem $\gamma > 0$: $(2Y_i - 1)\tilde{\mathbf{x}}_i^T \mathbf{w}^* > \gamma$) a všechny body mají $\|\tilde{\mathbf{x}}_i\| \leq R$, pak počet chyb (a tedy kroků s opravou) je nejvýše R^2/γ^2 . *Intuice:* Když data jdou oddělit přímkou, perceptron to v konečném čase najde. Čím větší odstup γ , tím rychleji.

Náčrt důkazu: sleduje se dvojí. (1) Skalární součin $(\mathbf{w}^{k+1})^T \mathbf{w}^* \geq n\gamma$ roste lineárně s počtem chyb. (2) Norma $\|\mathbf{w}^{k+1}\|^2 \leq nR^2$ roste nejvýš lineárně. Spojením $n^2\gamma^2 \leq \|\mathbf{w}\|^2 \leq nR^2$, odkud $n \leq R^2/\gamma^2$.

7.3 Limit perceptronu: XOR a AI winter *

Pozor: Perceptron umí jen lineárně separabilní úlohy. Funkci **XOR** (1 právě když se vstupy liší) *nedokáže* vyřešit — body nejdou oddělit jednou přímkou. Minsky & Papert (1969) to publikovali, což vyvolalo „AI winter“ (15 let útlumu).

Řešení: XOR vyřeší dva perceptrony (realizující AND a NOR) s dalším perceptronem nad nimi — tj. **vícevrstvá síť**. Problém byl ale *učení*: algoritmus pro jeden perceptron nešlo rozšířit na propojené neurony. Průlom přišel v 80. letech s *gradientním sestupem* a *zpětným šířením chyby*.

8 Neuronové sítě II: vícevrstvé sítě a backpropagation

8.1 Vícevrstvá síť (MLP) *

Vícevrstvá (dopředná) neuronová síť = neurony uspořádané do vrstev; výstupy jedné vrstvy jsou vstupy další. Při klasifikaci se říká **vícevrstvý perceptron (MLP)**. **Plně propojená** vrstva = do každého neuronu jdou všechny výstupy předchozí vrstvy. Vrstvy kromě poslední jsou **skryté**; počet vrstev = **hloubka**.

Matematicky: i -tá vrstva je funkce $f^{(i)}: \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$, kde každý neuron počítá $g(\mathbf{w}^T \mathbf{x} + w_0)$. Celá síť je **složení** vrstev:

$$f = f^{(l)} \circ f^{(l-1)} \circ \dots \circ f^{(1)}.$$

Intuice: Role skrytých vrstev: vytvářejí příznaky pro další vrstvy. Na rozdíl od jádrových metod (kde příznaky volíme ručně) si síť příznaky sama vymyslí při trénování. Hlubší síť = sofistikovanější příznaky.

Věta o univerzální aproximaci. Síť s *jednou* skrytou vrstvou (nelineární aktivace) umí aproximovat *libovolnou* spojitou funkci s libovolnou přesností. **Pozor:** V praxi by to chtělo obrovské množství neuronů a hrozí přeučení — proto se radši dělají *hlubší* sítě (více vrstev, méně neuronů na vrstvu). Problémem hloubky je ale učení.

8.2 Aktivační funkce *

Výstupní vrstva (podle typu úlohy):

- Regrese:** jeden neuron, žádná aktivace (identita): $\hat{Y} = \mathbf{w}^T \mathbf{h} + w_0$.
- Binární klasifikace:** jeden neuron se **sigmoidou**
 $\sigma(z) = \frac{1}{1 + e^{-z}}$; výstup je $\hat{\mathbb{P}}(Y=1 | \mathbf{x})$. Predikce 1, když $> 0,5$.
- Klasifikace do c tříd:** c neuronů se **softmaxem**
 $\text{softmax}_i(z) = \frac{e^{z_i}}{\sum_j e^{z_j}}$; výstupy jsou pravděpodobnosti tříd (sčítají se do 1). Predikce = $\arg \max$. *Intuice:* Softmax je „měkký argmax“ — diferencovatelná aproximace one-hot výběru největšího.

Skryté vrstvy (musí být nelineární a skoro všude diferencovatelné):

- ReLU:** $g(z) = \max(0, z)$. Nejpoužívanější; rychlá. Nevýhoda: pro $z < 0$ má nulový gradient („mrtvé neurony“).
- Leaky ReLU:** z pro $z \geq 0$, $0,01z$ jinak — nenulový gradient i v záporu.
- SELU:** škálovaná exponenciální; při vhodné inicializaci sama udržuje průměr/rozptyl mezi vrstvami.
- tanh:** $\frac{e^z - e^{-z}}{e^z + e^{-z}}$ — používala se před ReLU.

Pozor: Bez nelineární aktivace by složení vrstev bylo zase jen lineární zobrazení — síť by neuměla nic víc než lineární model.

8.3 Ztrátové a účelové funkce *

Ztrátová funkce $L(Y, \hat{Y})$ měří chybu v jednom bodě:

- Regrese: **kvadratická** $L = (Y - \hat{Y})^2$, případně $L_1: |Y - \hat{Y}|$.
- Binární klasifikace: **binární křížová entropie**
 $L = -Y \log \hat{p} - (1 - Y) \log(1 - \hat{p})$.
- c tříd: **kategorická křížová entropie**
 $L = -\sum_j \mathbf{1}_{Y=j} \log \hat{p}_j = -\log \hat{p}_Y$.

Intuice: Křížová entropie trestá hlavně to, když jsi sebejistý a pleteš se ($\hat{p} \rightarrow 0$ u správné třídy $\Rightarrow -\log \hat{p} \rightarrow \infty$).

Účelová funkce $J(\mathbf{w})$ = průměr ztráty přes trénovací data:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N L(Y_i, f(\mathbf{x}_i; \mathbf{w})).$$

Pro kvadratickou ztrátu je to **MSE** (střední kvadratická chyba). Minimalizujeme J gradientním sestupem.

8.4 Zpětné šíření chyby (backpropagation) *

Problém: jak spočítat gradient $\nabla_{\mathbf{w}} J$ pro síť, která je složením mnoha vrstev? **Odpověď: řetězové pravidlo.**

Připomenutí řetězového pravidla pro složení:

$\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)$. Pro vícehodnotové funkce $g: \mathbb{R}^m \rightarrow \mathbb{R}^n$ a $f: \mathbb{R}^n \rightarrow \mathbb{R}$:

$$\frac{\partial(f \circ g)}{\partial x_j}(\mathbf{x}) = \sum_{i=1}^n \frac{\partial f}{\partial g_i}(g(\mathbf{x})) \frac{\partial g_i}{\partial x_j}(\mathbf{x}).$$

Dopředný chod (forward pass): spočteme výstup sítě $f(\mathbf{x}; \mathbf{w})$ a hodnotu J . **Zpětný chod (backward pass):** gradient počítáme *od výstupní vrstvy zpět ke vstupu*, přičemž postupně násobíme a sčítáme parciální derivace jednotlivých vrstev (řetězové pravidlo, vrstva po vrstvě). Proto **zpětné šíření chyby (back-propagation)**.

Intuice: Backprop = chytré uspořádání výpočtu řetězového pravidla. Místo abychom derivovali vše znovu pro každou váhu, „protáhneme“ derivaci ztráty sítě pozpátku a opakovaně používáme mezivýsledky. Bez toho by byly hluboké sítě neúčinné.

Příklad (regrese, 2 vstupy $\rightarrow$ 2 neurony s aktivací $g \rightarrow$ 1 lineární neuron):

$$f(\mathbf{x}; \mathbf{w}) = w_1^{(2)} g(\mathbf{x}^T \mathbf{w}_1^{(1)} + w_{0,1}^{(1)}) + w_2^{(2)} g(\dots) + w_{0,2}^{(2)}.$$

Složky gradientu (kvadratická ztráta) se počítají řetězově, např.

$$\frac{\partial J}{\partial w_{1,1}^{(1)}} = \frac{1}{N} \sum_i 2(Y_i - f)(-1) w_1^{(2)} g'(\mathbf{x}_i^T \mathbf{w}_1^{(1)} + w_{0,1}^{(1)}) x_{i,1}.$$

Vidíme řetěz: derivace ztráty $\times$ váha výstupního neuronu $\times$ derivace aktivace $\times$ vstup.

Výpočetní graf. Výpočty (dopředné i zpětné) se reprezentují **výpočetním grafem**: vrcholy = proměnné, hrany = elementární operace ($+$, $\times$, aktivace). Zpětný chod prochází graf pozpátku a v každém uzlu lokálně aplikuje derivaci. (Tohle dělají knihovny jako PyTorch automaticky — „autograd“.)

Ke zkoušce: Vysvětlit MLP jako složení vrstev a roli skrytých vrstev (učené příznaky). Znat aktivace (ReLU, sigmoid, softmax) a kdy se která použije. Znat ztrátové funkce (MSE, křížová entropie). Slovně i na malém příkladu popsat backpropagation jako řetězové pravidlo počítané pozpátku.

9 Neuronové sítě III: trénování, optimalizace, regularizace

9.1 Dávkové (minibatch) učení *

Deterministické učení: gradient $\nabla_{\mathbf{w}} J$ počítáme z chyby přes celou trénovací množinu. Pro velká data je to pomalé a paměťově náročné (až neproveditelné).

Stochastické dávkové učení: v jednom kroku použijeme jen **dávku (batch)** o velikosti m a z ní spočteme *odhad* gradientu:

$$\tilde{J}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m L(Y_i, f(\mathbf{x}_i; \mathbf{w})).$$

Intuice: Místo přesného, ale drahého gradientu uděláme hodně zašuměných, ale levných kroků. Šum dokonce pomáhá utéct z mělkých minim. Typické velikosti dávky: mocniny 2 (16, 32, ..., 256). Dávka velikosti 1 = *online* učení.

Cyklus učení. Data *náhodně zamícháme* (důležité!) a rozdělíme na dávky velikosti m . Projdeme všechny dávky (po N/m krocích jsme prošli celá data) — tomu se říká **epocha**. Pak pokračujeme další epochou. Na konci epochy vyhodnotíme stav (např. validační chybu). Skončíme po daném počtu epoch nebo když je výkon dost dobrý.

9.2 Stochastický gradientní sestup (SGD) *

Nezákladnější optimalizátor:

1. Inicializuj $\mathbf{w}$, zvol **učicí parametr** ϵ (learning rate).
2. Opakuj: vezmi dávku, spočti $\tilde{\nabla} = \nabla_{\mathbf{w}} \tilde{J}(\mathbf{w})$, uprav $\mathbf{w} \leftarrow \mathbf{w} - \epsilon \tilde{\nabla}$.

Pozor: Volba ϵ je zásadní. *Velké* $\epsilon \rightarrow$ oscilace a divergence (skáčeme přes minimum). *Malé* $\epsilon \rightarrow$ pomalá konvergence, uvíznutí. Učicí parametr se často nechává **klesat** s iteracemi (zpočátku rychle, později jemně).

Inicializace vah. Náhodně (rovnoměrně/normálně), klíčové je správné **měřítko**. Oblíbená je **Xavier (Glorot)**:

$\text{Unif}(-\sqrt{6/(m+n)}, \sqrt{6/(m+n)})$, kde m vstupů a n neuronů vrstvy.

Intuice: Špatné měřítko $\Rightarrow$ signál buď exploduje, nebo vyhasne — viz problém gradientu níže.

9.3 SGD s hybností (momentum) *

Intuice: Analogie: kulička valící se z kopce má setrvačnost. Místo abychom se řídili jen aktuálním sklonem, držíme „rychlost“ a tlumíme oscilace. Newtonovsky: gradient je síla měnící rychlost v . Update:

$$v \leftarrow \mu v - \epsilon \tilde{\nabla}, \quad \mathbf{w} \leftarrow \mathbf{w} + v,$$

kde μ (parametr hybnosti) říká, kolik si pamatujeme z předchozí rychlosti. Pomáhá rychleji projít „dlouhú údolí“. (Existuje i *Nesterovova* varianta.)

9.4 Adaptivní učicí parametry: AdaGrad, RMSProp, Adam *

Chceme *různý* learning rate pro různé váhy a různé fáze učení.

- **AdaGrad:** každou váhu škáluje $1/\sqrt{\text{součet kvadrátů jejích historických gradientů}}$.
 $r \leftarrow r + \tilde{\nabla} \odot \tilde{\nabla}, \mathbf{w} \leftarrow \mathbf{w} - \frac{\epsilon}{\delta + \sqrt{r}} \odot \tilde{\nabla}$ ($\odot$ = po složkách). Nevýhoda: r roste donekonečna $\rightarrow$ krok časem vyhasne.
- **RMSProp:** místo celé historie *exponenciálně klouzavý průměr* kvadrátů gradientu: $r \leftarrow \rho r + (1 - \rho) \tilde{\nabla} \odot \tilde{\nabla}$. Stará historie vymizí, krok nevyhasne. (Často s hybností.)

- **Adam (adaptive moments**, dnes nejpoužívanější): kombinuje hybnost (1. moment s) i RMSProp (2. moment r), plus *korekci* pro počáteční iterace:

$$s \leftarrow \rho_1 s + (1 - \rho_1) \tilde{\nabla}, \quad r \leftarrow \rho_2 r + (1 - \rho_2) \tilde{\nabla} \odot \tilde{\nabla},$$

$$\hat{s} = \frac{s}{1 - \rho_1^t}, \quad \hat{r} = \frac{r}{1 - \rho_2^t}, \quad \mathbf{w} \leftarrow \mathbf{w} - \epsilon \frac{\hat{s}}{\delta + \sqrt{\hat{r}}}.$$

Pozor: Žádný optimalizátor není vždy nejlepší. Adam (resp. AdamW při L2 regularizaci) je dobrá výchozí volba. Metody 2. řádu (Newton, BFGS) využívají Hessovu matici, ale jsou výpočetně drahé a vzácně se používají.

9.5 Regularizace neuronových sítí *

Problém: sítě s mnoha parametry se snadno *přeučí* (naucí se trénovací data nazpaměť). (Lokální minima a sedlové body naopak u velkých sítí velký problém nejsou — bývají blízko globálního minima.)

(1) **Penalizace norem vah.** K J přidáme $\lambda \Omega(\mathbf{w})$:

- **L2:** $\Omega = \|\mathbf{w}^{(i)}\|_2^2 = \sum_j (w_j^{(i)})^2$ — tlačí váhy k nule (jako ridge).
- **L1:** $\Omega = \|\mathbf{w}^{(i)}\|_1 = \sum_j |w_j^{(i)}|$ — vynuluje některé váhy (řídí síť, jako Lasso).

(2) **Předčasné zastavení (early stopping)** — nejpoužívanější. Na konci každé epochy změř validační chybu; pamatuj si váhy s nejmenší dosud viděnou. Když se ℓ epoch nezlepší, zastav a vrať nejlepší váhy. *Intuice:* Trénink přerušíme dřív, než se síť začne přeučovat.

(3) **Dropout.** Při trénování *náhodně vynulujeme* část vstupů do vrstvy (každý nezávisle s pravděpodobností p), pro každou dávku znovu. Nevynulované se škálují $1/(1 - p)$ (*inverzní dropout*); při predikci se nic nenuluje. *Intuice:* Síť se nesmí spoléhat na jeden „chytrý“ neuron — informaci musí rozprostřít. Je to jako trénovat zároveň spoustu menších podsítí (analogie k baggingu) a při predikci je zprůměrovat. p bývá menší pro vstupy ($\sim 0,2$), větší pro skryté ($\sim 0,5$). Levné, univerzální, zvyšuje generalizaci (typicky pak potřeba o něco větší síť).

9.6 Problém gradientu v hloubce a dávková normalizace *

Problém: update děláme pro všechny váhy najednou; změnou vah v hlubších vrstvách měníme vstupy mělkých — celkový efekt může být úplně jiný, než čekáme. Příklad: triviální síť $f = xw_1w_2 \dots w_l$. Při stejných vahách w je $f = xw^l$ — pro malé w je relativní změna po kroku mizivá (bylo by třeba velké ϵ), pro velké w obrovská. **Mizející / explodující gradient.**

Dávková normalizace (batch normalization). Výstupy skryté vrstvy (vnitřní potenciály) přes dávku *standardizujeme* a pak (volitelně) lineárně přeškálujeme:

$$h'_{i,j} = \frac{h_{i,j} - \hat{\mu}_j}{\sqrt{\delta + \hat{\sigma}_j^2}}, \quad \text{nahradíme } h_{i,j} \rightarrow \gamma_j h'_{i,j} + \beta_j,$$

kde $\hat{\mu}_j, \hat{\sigma}_j^2$ jsou průměr/rozptyl přes dávku a γ_j, β_j jsou *nové učené parametry*. Při predikci se místo statistik dávky použijí hodnoty nasbírané exponenciálním klouzavým průměrem při trénování. *Intuice:* Standardizace narovná měřítka mezi vrstvami (gradient se chová předvídatelně). Přidaná lineární transformace γ, β vrátí síti volnost ve střední hodnotě a rozptylu — ale teď ji řídí *přímo* dva parametry, což gradientní sestup snadno ovládne (jiná, lepší dynamika). Bias neuronů se pak vynechává (byl by redundantní). Přesně proč to funguje, není zcela jasné, ale empiricky moc pomáhá.

Ke zkoušce: Rozdíl deterministické vs. dávkové učení; co je epocha a proč míchat data. Napsat SGD a vysvětlit vliv ϵ . Vysvětlit hybnost a princip Adamu (1. a 2. moment). Tři regularizace: L1/L2, early stopping, dropout (a proč dropout pomáhá). Problém mizejícího gradientu a myšlenku batch norm.

10 Konvoluční a rekurentní sítě (CNN, RNN)

10.1 Konvoluce *

Konvoluční sítě (CNN) jsou dopředné sítě pro data s *maticovou/tenzorovou* strukturou — obrázky (2D), časové řady (1D). Aspoň v jedné vrstvě počítají **konvoluci**.

Spojité konvoluce dvou funkcí: $(f * g)(t) = \int f(\tau)g(t - \tau) d\tau$. Je komutativní. Používá se k *vyhlazení* f jádrem g (typicky hustota —

rovnoměrná, Gaussovská). Hezká vlastnost: $(f * g)' = f * g'$ (derivace vyhlazené funkce = konvoluce s derivací jádra $\Rightarrow$ detekce hran) a **ekvivariance vůči posunu** (posunu-li vstup, posune se stejně i výstup).

Diskrétní konvoluce (posloupnosti): $(x * w)(t) = \sum_i x(i)w(t-i)$. Vyhlažování (jádro samé jedničky) nebo detekce hran (jádro $-1, 1$).

Konvoluce v sítích. Jádro K jsou *učené parametry*. Pro 2D obrázek I a jádro K se v praxi počítá **křížová korelace** (v knihovnách zvaná „konvoluce“ — vynechává se otáčení jádra, které pro učené parametry nepotřebujeme):

$$(I * K)(i, j) = \sum_{m=1}^{k_1} \sum_{n=1}^{k_2} I(i+m-1, j+n-1) K(m, n).$$

Výstup má rozměry $(p_1 - k_1 + 1) \times (p_2 - k_2 + 1)$. *Intuice:* Jádro = malé „okénko“ (filtr), které posouváme po obrázku a v každé pozici počítáme vážený součet pixelů pod ním. Naučené filtry detekují hrany, rohy, textury...

10.2 Konvoluční vrstva a její principy *

Data jsou tenzory s **kanály** (např. RGB obrázek = tenzor $(3, 28, 28)$, „channel first“). Výstupní kanál j :

$$O_j = b_j + \sum_{\ell=1}^{c_{in}} I_{\ell} * K_{j,\ell},$$

tj. konvoluce přes všechny vstupní kanály se sečtou a přidá se bias. Parametry na jeden výstupní kanál: $c_{in} \cdot k_1 \cdot k_2 + 1$.

Dvě klíčové vlastnosti (proč mají CNN málo parametrů):

- **Omezené spojení (sparse connectivity):** jádro je menší než obrázek, takže každý výstupní bod závisí jen na malé **oblasti vnímání (receptive field)** vstupu. (Inspirováno vizuální kúrou V1 — Hubel & Wiesel.) S hloubkou se oblast vnímání rozšiřuje.
- **Sdílení parametrů:** stejné jádro se používá na všech pozicích. Konvoluční vrstva = plně propojená vrstva s *lokálně omezenými* a *svázanými* parametry.

Intuice: Plně propojená vrstva by pro obrázek měla šílené množství vah. CNN jich má řádově méně — proto mohou být hluboké — a navíc detekuje vzor bez ohledu na to, kde v obrázku je.

Další stavební bloky.

- **Aktivace (ReLU)** po konvoluci.
- **Pooling (sdružování):** zmenší výstup a přidá invarianci vůči malým posunům. **Max pooling** bere maximum z okénka; existuje i average pooling.
- **Padding (doplnění okrajů):** přidá hodnoty (typ. nuly) kolem vstupu, aby se rozměry nezmenšovaly.
- **Stride (krok posunu):** posun okénka o víc než 1 $\rightarrow$ menší výstup (downsampling). Výstup pak $\lfloor (p - k) / s \rfloor + 1$.

CNN typicky: několik konvolučních vrstev + pár plně propojených na konci. Trénují se nejlépe na GPU. Skvělé na obrázky i časové řady pevné délky.

10.3 Rekurentní síť (RNN) *

Problém: co s posloupnostmi *různé* délky $x^{(1)}, \dots, x^{(\tau)}$ (věty, časové řady)? Plně propojené ani konvoluční síť (pro pevnou délku) nestačí.

Řešení — rekurence. Udržujeme **skrytý stav** $h^{(t)}$, který se s časem vyvíjí na základě nového vstupu a předchozího stavu:

$$h^{(t)} = g_1(U^T x^{(t)} + W^T h^{(t-1)} + w_0), \quad o^{(t)} = g_2(V^T h^{(t)} + v_0).$$

Intuice: $h^{(t)}$ je „paměť“ — souhrn relevantní informace z $x^{(1)}, \dots, x^{(t)}$. Síť má smyčku: výstup minulého kroku ovlivní příští. Výstupem může být celá posloupnost $o^{(1)}, \dots, o^{(\tau)}$ (překlad) nebo jen poslední $o^{(\tau)}$ (klasifikace).

Trénování — rozbalení (unfolding): smyčku „rozvineme“ do hlubokého výpočetního grafu přes čas a aplikujeme backprop (backpropagation through time). **Pozor:** RNN mají problémy jako velmi hluboké síť (mizející gradient) a špatně zvládají **dalekodohové závislosti** (informaci rozprostřenou daleko od sebe). Řeší to složitější jednotky, hlavně **LSTM** (long short-term memory).

10.4 Závěrečné poznámky k sítím *

- **Transformery** (self-attention) — dnes základ NLP, viz další kapitola.
- **Autoenkodéry** — komprese informace do menšího prostoru / opravy chyb.
- **Generativní modely** — síť se učí transformovat normální rozdělení na cílové (generování obrázků).
- **Data augmentation** — při málu dat vyrobíme víc trénovacích příkladů poruchami (otáčení, posun, škálování obrázků).
- **Adversariální útoky** — drobné, okem neznatelné změny vstupu zcela změni výstup; brání se adversariálním trénováním.
- **Transfer learning** — vezmeme předtrénovanou síť a jen ji doučíme na svou úlohu.
- Knihovny: **TensorFlow**, **PyTorch**.

Ke zkoušce: Vysvětlit konvoluci jako posuvné okénko a dvě vlastnosti CNN: omezené spojení (receptive field) a sdílení parametrů — a proč to šetří parametry. Znát pooling, padding, stride. Popsat RNN přes skrytý stav a vědět o problému dalekodohových závislostí (LSTM).

11 Zpracování přirozeného jazyka (NLP)

11.1 O co jde a statistický přístup *

NLP = strojové zpracování přirozeného jazyka. Tři hlediska jazyka: **syntax** (jak jsou slova uspořádána — gramatika), **sémantika** (význam), **pragmatika** (vztah ke komunikační situaci). ML se nejčastěji zabývá významem.

Strukturalismus vs. statistika. Snaha popsat jazyk *pravidly* (strukturalismus) ztroskotala — jednotlivá pravidla fungují, ale jejich kombinace vyžaduje pravidla pro pravidla... **Statistický přístup** (= strojové učení) bere texty prostě jako data a dnes vyhrál prakticky na všech frontách.

Korpus = (velký) soubor textů, zdroj dat. Příklady: Český národní korpus (> 16 mld. slov), ÚFAL/LINDAT, NLTK; pro moderní modely **Common Crawl** (petabajty webu), OSCAR, Wikipedia dumps, Hugging Face Datasets. **Pozor:** Webové korpusy obsahují duplicity, šum a spam — před použitím se čistí a deduplikují.

11.2 Bag of words *

Korpus $C = \{d_1, d_2, \dots\}$ (dokumenty). **Bag of words:** dokument bereme jen jako *hromádku slov* (pořadí ignorujeme). Se slovníkem D reprezentujeme korpus **maticí**: řádek = dokument, sloupec = slovo, hodnota = počet výskytů slova v dokumentu. Dokumenty i slova jsou pak *vektory*.

Intuice: Stačí to na klasifikaci dokumentů, detekci spamu. Nestačí tam, kde záleží na pořadí (strojový překlad).

Problémy bag of words:

1. Matice je obrovská (v češtině spousta různých slov).
2. Některá slova jsou všude (být, spojky, předložky), jiná podreprezentovaná.
3. Matice je velmi **řidká** (skoro samé nuly).

11.3 Předzpracování: stop slova a lemmatizace *

- **Stop slova** — slova bez významové hodnoty (být, mít, spojky, předložky). Obvykle se odstraní (i interpunkce). **Pozor:** Pozor na kontext: smajlíky mohou nést sentiment; slovo „gól“ je dobrý indikátor tématu fotbal — neodstraňovat slepě podle frekvence.
- **Lemmatizace** — každé slovo nahradíme **lemmatem** (základní tvar: 1. pád, infinitiv). U flektivních jazyků (čeština) zásadně snižá dimenzi: „koloběžka/koloběžkou“ $\rightarrow$ „koloběžka“. Netriviální úloha; pro češtinu např. **MorphoDiTa** (ÚFAL).

11.4 Tf-idf *

Místo holého počtu výskytů chceme vyjádřit *váhu (důležitost)* slova. **Tf-idf** = součin dvou čísel:

- **tf (term frequency)** — jak často je slovo w v dokumentu d . Varianty: indikátor 0/1, počet $f_{w,d}$, $\log(1 + f_{w,d})$, normalizovaná verze.
- **idf (inverse document frequency)** — jak je slovo v rámci korpusu vzácné: $\text{idf}(w, C) = \log \frac{|C|}{|\{d \in C : w \in d\}|}$ (případně s +1 ve jmenovateli).

$$\text{tf-idf}(w, d) = \text{tf}(w, d) \cdot \text{idf}(w, C).$$

Intuice: Slovo je důležité, když je *časté v tomto dokumentu*, ale *vzácné v korpusu*. Slova všude (vysoké idf u nuly) se utlumí, charakteristická slova se zvýrazní. **Pozor:** tf závisí jen na dokumentu, ale idf vyžaduje projít celý korpus — výpočet může být náročný. (scikit-learn: TfidfVectorizer.)

11.5 Klasifikace dokumentů *

Postup: rozdělíme korpus na trénovací/testovací; dokumenty převedeme na vektory (TfidfVectorizer, slova = příznaky); *stejný* naučený vektorizér použijeme na test; a pak na „tabulková“ data pustíme klasifikátor. Tisíce příznaků (= slov), takže se hodí: **logistická regrese, SVC, hlavně naivní Bayes**. Dnes vévodí neuronové sítě — ty ale nepoužívají bag-of-words, nýbrž vektorovou reprezentaci slov.

11.6 Moderní NLP: embeddings a Transformers *

Vektorová reprezentace slov (word embeddings). V bag-of-words má každé slovo vlastní sloupec a slova jsou *ortogonální* — model neví, že „pes“ a „štěně“ spolu souvisí. Moderní přístup: každé slovo = *hustý* vektor ve stovkách dimenzí, kde podobná slova leží blízko. Učí se z velkých korpusů podle **distribuční hypotézy** (slova v podobných kontextech mají podobný význam).

- **word2vec** (2013): vektor se naučí predikcí kontextu (skip-gram) nebo slova z kontextu (CBOW).
- **GloVe, fastText** — fastText umí podslova (n-gramy znaků, vhodné pro češtinu).
- Funguje aritmetika analogii: $\text{král} - \text{muž} + \text{žena} \approx \text{královna}$.

Pozor: Slabina statických embeddings: „kohoutek“ má jeden vektor bez ohledu na význam (vodovodní vs. zvíře).

Self-attention a Transformers (Vaswani et al., 2017, „Attention is All You Need“) — základ dnešních modelů (BERT, GPT, T5, LLaMA).

- Klíč je **self-attention**: pro každé slovo se spočte, „kolik pozornosti“ věnovat ostatním slovům, a podle toho se jeho reprezentace upraví $\Rightarrow$ **kontextový embedding** (kohoutek má jiný vektor podle věty).
- Mechanismus pracuje s trojicemi **Query, Key, Value** (Q, K, V): váhy pozornosti = softmax ze *škálovaného skalárního součinu* Q a K , ty se aplikují na V .
- **Výhody**: celá sekvence se zpracuje *paralelně* (na rozdíl od RNN $\rightarrow$ efektivní na GPU), dobře zachytí dlouhé závislosti.
- **Pretrain + fine-tuning**: model se předtrénuje na obrovských korpusech a pak doladí na konkrétní úlohu s málem anotovaných dat. V praxi často stačí vzít předtrénovaný model z Hugging Face a doučit poslední vrstvu (klasifikace, NER, překlad, sumarizace, QA).

Ke zkoušce: Popsat bag of words a jeho tři problémy. Vysvětlit stop slova a lemmatizaci (proč u češtiny). Odvodit tf-idf a vysvětlit intuici (časté v dokumentu, vzácné v korpusu). Vědět, čím se liší embeddings od bag-of-words a co dělá self-attention (kontextový embedding, $Q/K/V$, paralelní zpracování).

12 Posilované učení (RL)

12.1 Co to je *

Posilované učení (reinforcement learning) je *třetí paradigma* ML (vedle supervizovaného a nesupervizovaného), nejbliž obecně představě umělé inteligence. **Agent** koná **akce** v **prostředí**, které je v nějakém **stavu**; po akci dostane **odměnu (reward)** a prostředí typicky změní stav. Cíl: naučit agenta chovat se tak, aby *maximalizoval celkovou (i budoucí) odměnu*.

Intuice: Nikdo agentovi neříká „udělej tohle“. Sám musí *zkoušením* zjistit, které akce se vyplácí — a brát ohled nejen na okamžitou odměnu, ale i na všechny budoucí. Prostředí i odměny mohou být *náhodné* a měnit se v čase. Úspěchy: AlphaGo Zero (Go, 2017), AlphaStar (StarCraft II, 2019), řízení chlazení datacenter Google, ladění LLM (např. ChatGPT).

12.2 Explorace vs. exploatace *

Ústřední dilema RL. **Explorace** = zkoušení nových akcí (sbírání informace). **Exploatace** = využití toho, co už víme (volba zatím nejlepší akce).

Příklad (cesta do školy). 3 možnosti (pěšky/kolo/metro), každá dorazí včas s jinou (neznámou) pravděpodobností (30%/50%/90%), cíl: max. včasných příchodů za 30 dní.

- Jen exploatovat po krátké exploraci (3 dny, pak pořád to, co vyšlo nejlíp) je riskantní — snadno se „upneme“ na špatnou volbu ($\approx 20,9$ včas v průměru).
- Jen pořád střídát (čistá explorace) $\rightarrow \approx 17$ včas.
- Kombinace (15 dní zkoušej, 15 dní exploatuj nejlepší) $\rightarrow \approx 21,8$ včas.

Pozor: Ani jedno samo nestačí. Agent musí zkoušet různé akce a zároveň progresivně preferovat ty nejlepší. Neexistuje triviální optimální řešení — algoritmy obě činnosti nějak kombinují.

12.3 k-ruký bandita *

Jednoruký bandita = výherní automat s pákou (jedna akce, náhodná odměna). **k-ruký (víceruký) bandita** = k automatů (akcí), každý s jiným rozdělením odměn. (Bez stavu prostředí — to přidá až MDP.)

Hodnota akce. Akci v čase t značíme A_t , odměnu R_t . **Hodnota akce** a :

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a] \quad (\text{očekávaná odměna za akci } a).$$

Kdybychom q_* znali, volíme vždy $\arg \max_a q_*(a)$. Neznáme ji, takže si ji **odhadujeme** výběrovým průměrem získaných odměn:

$$Q_t(a) = \frac{\sum_{\tau < t} R_\tau \mathbf{1}_{A_\tau = a}}{\sum_{\tau < t} \mathbf{1}_{A_\tau = a}}.$$

Ze zákona velkých čísel $Q_t(a) \rightarrow q_*(a)$ s počtem voleb.

Inkrementální update (úspora paměti). Průměr nemusíme počítat z celé historie:

$$Q_{n+1} = Q_n + \frac{1}{n} (R_n - Q_n).$$

Obecné pravidlo:

NovýOdhad $\leftarrow$ StarýOdhad + KrokVel $\cdot$ (Cíl – StarýOdhad), kde velikost kroku nemusí být $1/n$.

12.4 Algoritmy pro k-rukého banditu *

Hladový výběr (greedy): $A_t = \arg \max_a Q_t(a)$. Jen exploatuje — přestane zkoušet, což je špatné, dokud nemáme dobré odhady.

ε -hladový výběr: s pravděpodobností $1 - \varepsilon$ hladově, s pravděpodobností ε *náhodná* akce. Tím se každá akce občas vyzkouší $\Rightarrow Q_t(a) \rightarrow q_*(a)$. Algoritmus:

1. Inicializuj $Q(a) = 0, N(a) = 0$; zvol ε .
2. Opakuj: $A \leftarrow (\arg \max_a Q(a) \text{ s pst. } 1 - \varepsilon, \text{ jinak náhodně})$;
 $R \leftarrow \text{bandit}(A); N(A)++; Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$.

Intuice: Větší ε = víc explorace (ale i v pozdní fázi rušíme optimální volby). Menší ε = víc exploatace.

Nestacionární případ (rozdělení se mění v čase): dej větší váhu novějším odměnám — konstantní krok $\alpha \in (0, 1]$ místo $1/n$:

$$Q_{n+1} = Q_n + \alpha(R_n - Q_n) = \alpha R_n + (1 - \alpha)Q_n,$$

což je *exponenciální klouzavý průměr*. (Konvergence k q_* dle ZVČ vyžaduje $\sum \alpha_n = \infty$ a $\sum \alpha_n^2 < \infty$; $1/n$ to splňuje, konstantní α ne — ale v nestacionárním případě stejně není k čemu konvergovat.)

Optimistické počáteční hodnoty: nastav $Q(a)$ na vysoké hodnoty $\Rightarrow$ agent ze začátku zklamaně zkouší všechny akce (přirozená explorace na startu).

UCB (Upper Confidence Bound): systematická explorace místo náhodně. Volíme

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right].$$

Odmocnina je míra *nejistoty* odhadu: málo zkoušená akce má velký bonus (zkus mě!), často zkoušená malý. *Intuice:* UCB kombinuje „ber aktuálně nejlepší“ se „sniž nejistotu“ — preferuje slibné a zároveň málo prozkoumané akce.

12.5 Markovský rozhodovací proces (MDP) *

Bandita nemá *stav* prostředí, který se mění po akci. **MDP** to přidává. V kroku t agent vidí stav S_t a odměnu R_t , zvolí akci A_t , načež dostane R_{t+1} a nový stav S_{t+1} .

Dynamika. S = množina stavů, A = množina akcí, a

$$\mathbb{P}(S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a) = p(s', r \mid s, a).$$

MDP je trojice (S, A, p) . **Markovská vlastnost:** přechod závisí jen na *aktuálním* stavu a akci, ne na historii, jak jsem se do stavu dostal.

Cíl — vážená odměna (discounted return). Pro **discount** $0 \leq \gamma \leq 1$:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

maximalizujeme očekávanou $\mathbb{E}G_t$. *Intuice:* $\gamma < 1$ znamená „budoucí odměny mě zajímají, ale ty bližší o trochu víc“ (a zaručí konečnost součtu).

Kam dál (jen pojmy). Moderní metody aproximují (např. neuronovou sítí) **action-value funkci** $q(s, a) = \mathbb{E}[G_t \mid S_t = s, A_t = a]$ při optimální strategii. Řeší se i případ, kdy agent vidí stav jen *částečně* (pozorování místo plného stavu).

Ke zkoušce: Vysvětlit schéma RL (agent/prostředí/stav/akce/odměna) a dilema *explorace* vs. *exploitace*. Napsat ϵ -hladový algoritmus a inkrementální update Q . Vysvětlit UCB (proč odmocnina = nejistota) a optimistické počáteční hodnoty. Definovat MDP (S, A, p) , Markovskou vlastnost a discounted return G_t s rolí γ .

Závěrečné shrnutí kurzu

Co umět propojit: **(1)** báze funkce $\rightarrow$ duální reprezentace $\rightarrow$ jádrový trik $\rightarrow$ SVM (jeden myšlenkový tok). **(2)** Redukce dimenze: PCA (lineární, max. rozptyl, vlastní vektory) vs. LLE/manifold (nelineární, lokální) vs. LDA (řízená třídami). **(3)** Pravděpodobnostní klasifikace: MAP + Bayes $\rightarrow$ naivní Bayes $\rightarrow$ generativní (GDA/QDA/LDA) vs. diskriminativní, a most LDA $\leftrightarrow$ logistická regrese. **(4)** Neuronové sítě: perceptron $\rightarrow$ MLP $\rightarrow$ backprop $\rightarrow$ optimalizace (SGD/Adam) + regularizace (dropout, early stopping, batch norm) $\rightarrow$ speciální architektury (CNN, RNN, Transformer). **(5)** NLP jako aplikace (bag-of-words/tf-idf $\rightarrow$ embeddings $\rightarrow$ Transformer). **(6)** RL jako samostatné paradigma (bandita $\rightarrow$ MDP).

Hodně štěstí u zkoušky!